

TAREQO

Décisions d'architecture — TAREQO ERP

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Pourquoi la Clean Architecture ? 4

Comment chaque dossier est utilisé 4

Pourquoi des contrats (interfaces abstraites) dans le domaine ? 5

Pourquoi la validation des prix vit dans le cas d'usage (pas dans l'UI) ? 5

Pourquoi commencer par une liste en mémoire puis migrer vers SQLite ? 6

Pourquoi SQLite s'exécute-t-il de façon synchrone sur l'isolate principal (et pourquoi NE PAS migrer vers des isolates maintenant) ? 6

Pourquoi Riverpod pour la gestion d'état ? 7

Comment Riverpod parle au domaine sans violer la règle de dépendance 7

Pourquoi une vente stocke le prix au moment de la vente (snapshot) ? 8

Comment garantir que le stock ne passe jamais négatif (ProcessSaleUseCase) 8

Comment les couches communiquent sans couplage 9

Pourquoi des quantités entières en MILLI-UNITÉS (W11) ? 9

Pourquoi des lots FIFO avec péremption ? 9

Pourquoi le format balance `20PPPPPWWWWW` ? 10

Pourquoi l'échelle typographique suit Apple HIG et Material (34/28/22/20/17/15/13/11) ? 10

L'échelle complète (8 niveaux, mappés aux rôles Material) 10

Règles d'or appliquées 11

Pourquoi une entité financière séparée de la vente ? 12

Contrôle du crédit (« fiado ») et fonds de roulement 12

Structure 12

Impact sur le fonds de roulement 13

Pourquoi l'authentification est-elle la première étape du modèle SaaS ? 13

Comment la session reste active (persistance de session) 13

Devise et langue du tenant (présentation) 14

Stockage sécurisé : pourquoi ne pas utiliser `shared_preferences` pour les données sensibles ? 14

Différence technique entre stockage normal et sécurisé 14

Pourquoi c'est vital pour protéger le compte du commerçant 15

Comment le service a été conçu 15

Frontière de responsabilité (un seul backend de stockage, pas de duplication) 15

Particularité de plateforme (précision technique) 15

Dépendances initiales 15

Génération de code (Freezed) 16

> Ce document enregistre le « pourquoi » derrière les principales décisions techniques de TAREQO ERP : chaque choix d'architecture, de modélisation et de sécurité est justifié ici. Pour la vue d'ensemble et le catalogue complet, voir les documents 01-overview à 11-multitenancy.

Pourquoi la Clean Architecture ?

Nous avons adopté la Clean Architecture pour TAREQO parce qu'un ERP est, par nature, un système durable avec des règles métier complexes et en constante évolution : les modules tels que le stock, les ventes, les achats, les finances et les rapports doivent être développés, testés et remplacés indépendamment, sans que les changements d'un module n'aient des effets secondaires sur les autres.

En divisant le code en trois couches — domaine (entités, contrats de repository et cas d'usage, libres de toute dépendance framework), données (implémentations concrètes des sources de données distantes et locales, comme Supabase et SQLite) et présentation (écrans, widgets et état via Riverpod) — l'architecture garde les règles métier au centre du système, indépendantes des détails d'infrastructure. Cela apporte quatre gains directs :

1. Testabilité — les cas d'usage peuvent être testés avec des mocks, sans serveur ni base de données ;
2. Remplaçabilité — la source de données peut migrer d'une API REST vers une base locale sans toucher au domaine ;
3. Maintenabilité — chaque développeur travaille sur des modules isolés avec des contrats clairs entre les couches ;
4. Évolutivité d'équipe — les interfaces du domaine servent de « contrats » qui permettent le travail en parallèle.

Comment chaque dossier est utilisé

```
lib/
 main.dart # Bootstrap : ProviderScope (Riverpod) + GetIt
 app.dart # Widget racine (MaterialApp, thème, locale/devise du tenant)
 core/ # Préoccupations transversales (n'appartiennent à aucune couche)
  config/ # Configuration (Supabase, préférences du tenant)
  database/ # Ouverture SQLite, schéma et transactions (Unit of Work)
  di/ # Service Locator (GetIt) — registre central des dépendances
  i18n/ # Clés de traduction et helpers de localisation
  security/ # Stockage sécurisé (Keychain/Keystore)
  theme/ # Thème Material 3 de l'app
  utils/ # Helpers génériques (formatage devise/date, ids...)
 domain/ # CŒUR : règles métier pures, aucune dépendance externe
  entities/ # Entités immuables (Product, Sale, Tenant...) via Freezed
  repositories/ # Contrats de persistance abstraits (ex. IProductRepository)
  usecases/ # Cas d'usage — orchestrent les règles métier
 data/ # Implémentations concrètes de ce que définit le domaine
  datasources/
   remote/ # Sources de données distantes (Supabase/REST)
   local/ # Sources de données locales (SQLite, en mémoire)
  models/ # DTO sérialisables — seulement quand le format API diverge du domaine
  repositories/ # Implémentations des interfaces du domaine
  services/ # Services d'intégration (comptes d'employés, processeurs atomiques)
 presentation/ # Tout ce que l'utilisateur voit et avec quoi il interagit
```

```
controllers/ # État : providers/notifiers Riverpod
screens/ # Écrans par module (auth, dashboard, products, sales, finance...)
widgets/ # Widgets partagés (KpiCard, EmptyState, StatusChip, AppSheet...)
```

> Règle de dépendance : le flux de dépendances pointe toujours vers l'intérieur. presentation → domain (via les contrats), data → domain (implémente les contrats), et domain ne sait rien des autres couches.

Pourquoi des contrats (interfaces abstraites) dans le domaine ?

Dans le domaine, IProductRepository est déclaré comme une interface abstraite — uniquement les signatures de méthodes (getAll, add) — pas comme une classe concrète. Parce que :

1. Inversion de dépendance (DIP) : le domaine ne peut pas connaître les détails d'infrastructure. La classe concrète (ex. ProductRepositoryImpl, avec SQLite ou Supabase) vit dans la couche data et *implémente* le contrat. Les règles métier dépendent d'une abstraction, jamais d'une implémentation.
2. Découplage et remplaçabilité : la source de données peut passer d'une API REST à une base locale (ou l'inverse) sans changer une seule ligne du domaine ni de l'UI — il suffit de créer une autre classe implémentant le même contrat.
3. Testabilité : dans les tests de cas d'usage, on injecte un *mock/fake* de IProductRepository (pas de serveur, pas de base), isolant la logique métier et gardant les tests rapides et déterministes.
4. Contrat d'intégration : l'interface sert d'accord entre les couches/équipes — data sait exactement quoi implémenter, presentation sait exactement quoi consommer, permettant le travail en parallèle.

Contrairement à l'héritage (où une classe concrète « est un » contrat), l'interface ne déclare que ce dont le système a besoin, laissant le comment à la couche de données.

Pourquoi la validation des prix vit dans le cas d'usage (pas dans l'UI) ?

AddProductUseCase valide que salePrice >= purchasePrice avant de persister. Cette règle a été délibérément placée dans le cas d'usage — pas sur l'écran d'inscription :

1. Les règles métier appartiennent au domaine. L'UI n'est que de la présentation. Si la validation vivait sur l'écran, chaque client (mobile, web, desktop) devrait la réimplémenter — et tout changement de règle exigerait de changer chaque écran.
2. Centralisation et cohérence. Chaque point d'entrée (formulaire, import, future API) passe par le même cas d'usage. Cela empêche un produit invalide d'entrer dans le système par une route qui ne validait pas.
3. Intégrité des données. La validation se produit aussi près des données que possible, garantissant que l'état du système ne contient jamais un produit dont le prix de vente est inférieur au prix d'achat — quelle que soit l'origine des données.
4. Testabilité. La règle est testée isolément avec des tests unitaires (cas d'usage + fake de repository), sans UI ni widgets — rapide et déterministe.

5. Réutilisation multi-plateforme. Le même cas d'usage sert l'UI Flutter, les scripts de migration et une future API REST.

En pratique, l'UI reste mince : elle n'attrape que l'exception levée par le cas d'usage et affiche le message à l'utilisateur.

Pourquoi commencer par une liste en mémoire puis migrer vers SQLite ?

ProductRepositoryImpl a commencé avec un datasource en mémoire — un choix délibéré :

1. Les règles métier d'abord. Sans configuration de base, migrations ni pilotes, nous avons validé l'architecture et la logique du domaine avant d'investir dans l'infrastructure.
2. Itération de bout en bout rapide. UI → cas d'usage → repository → datasource a fonctionné dès le premier jour, démontrant le flux ERP complet.
3. Pas d'engagement prématuré. Nous ne décidons entre SQLite, Hive, drift ou une API que lorsque nous connaissons les vraies exigences (requêtes, concurrence, synchronisation) — évitant un engagement prématuré envers une technologie.
4. Tests déterministes. Les données en mémoire gardent les tests rapides, prévisibles et indépendants de l'environnement.

La migration SQLite a été réalisée en suivant l'inversion de dépendance : des implémentations *SqliteDataSourceImpl ont été créées pour les mêmes contrats (SQL paramétré avec package:sqlite3), et le ServiceLocator a échangé les enregistrements — aucun changement dans le domaine, les cas d'usage, les repositories ni l'UI.

```
// lib/core/di/service_locator.dart — persistance active
final database = await AppDatabase.openAppDatabase();
if (database != null) {
 _registerSqlitePersistence(database); // SQLite (plateformes natives)
} else {
 _registerInMemoryPersistence(); // repli : web / échec de lib native
}
```

Détails d'implémentation :

- `lib/core/database/app_database.dart` — ouvre tareqo.db dans getApplicationDocumentsDirectory(), crée le schéma (CREATE TABLE IF NOT EXISTS avec CHECK de non-négativité comme garde finale), ne seed jamais de données (la base démarre vide — uniquement des données réelles) et expose runInTransaction (BEGIN/COMMIT/ROLLBACK) pour les écritures atomiques (ex. vente + articles).
- Les ventes sont persistées dans des tables normalisées (sales + sale_items) dans une seule transaction.
- Les enums sont persistées par nom ; les dates en millis epoch ; les booléens en 0/1.
- Web : le sqlite3 natif ne s'exécute pas dans le navigateur (dart:ffi n'existe pas), donc AppDatabase utilise un import conditionnel et l'app retombe sur les datasources en mémoire (mêmes contrats — l'UI reste identique).
- Tests : sur la VM Windows, package:sqlite3 retombe lui-même sur la winsqlite3.dll système, donc les tests s'exécutent sans configuration manuelle.

Pourquoi SQLite s'exécute-t-il de façon synchrone sur l'isolate principal (et pourquoi NE PAS migrer vers des isolates maintenant) ?

Décision (audit sept. 2026, point M6) : conserver les requêtes SQLite synchrones sur l'isolate principal. La migration vers IsolateDatabase/SynchronousDatabase a été évaluée et écartée à ce stade — ce n'est pas un problème actuel, c'est une optimisation prématurée à risque élevé :

1. Contrats 100 % synchrones. Toutes les interfaces locales (*LocalDataSource) et l'IUnitOfWork sont synchrones par conception ; les datasources en mémoire (repli web) et SQLite implémentent les mêmes contrats. Migrer vers des isolates exigerait de rendre asynchrones toutes les interfaces, implémentations, cas d'usage et repositories qui les consomment — un changement transversal dans le chemin de l'argent (ventes, crédits, achats, clôture de caisse), récemment stabilisé, sans changement de comportement visible.
2. Transactions réentrantes liées à une connexion unique. beginTransaction/commitTransaction/ rollbackTransaction utilisent un compteur statique de profondeur avec l'invariant documenté d'une connexion et d'un IUnitOfWork par processus. Sous isolates, cette sémantique devrait être redessinée de zéro (messages/ports) — un nouveau risque de corruption de transaction.
3. Le volume de données ne le justifie pas. Un POS à boutique unique (l'unité du produit) compte des milliers, pas des millions, de lignes ; les requêtes SQLite locales prennent des microsecondes à quelques millisecondes. Le point le plus coûteux (migrations v1–v6) s'exécute une seule fois au démarrage, déjà versionné par PRAGMA user_version.
4. Quand y revenir. Déclencheurs pour rouvrir le sujet : (a) télémétrie (Sentry) montrant des frames saccadées par SQL ; (b) rapports lourds agrégeant un long historique ; (c) multi-boutique avec un gros volume par appareil. Dans ces cas, migrer couche par couche (datasource par datasource) en suivant le même processus documenté de remplacement de persistance.

Pourquoi Riverpod pour la gestion d'état ?

La couche de présentation utilise Riverpod (Notifier/FutureProvider/StreamProvider) pour gérer l'état :

1. Réactivité déclarative. L'écran déclare ce qu'il surveille (ref.watch(productsProvider)) et Riverpod ne reconstruit que les widgets affectés — pas de setState manuel, d'InheritedWidget ou de streams faits main.
2. AsyncValue intégré. Les providers asynchrones exposent l'état comme AsyncValue (loading / error / data), rendant la gestion chargement/erreur/réessai déclarative et cohérente entre les écrans.
3. Composition de providers typée. Les providers peuvent dépendre d'autres providers, formant un graphe typé avec portée contrôlée via ProviderScope et *auto-dispose* quand on le souhaite.
4. Testabilité. Dans les tests, il suffit de surcharger les providers (overrideWithValue/overrideWith) — sans toucher à GetIt ni aux datasources.

Comment Riverpod parle au domaine sans violer la règle de dépendance

Le flux de dépendances pointe toujours vers l'intérieur :

```
ProductListScreen (widget)
 ↓ ref.watch
productsProvider (provider)
 ↓ ref.read
getProductsUseCaseProvider → GetProductsUseCase (domain)
 ↓
IProductRepository (contrat du domaine)
 ↓ (implémentation injectée via GetIt au bootstrap)
ProductRepositoryImpl → ProductLocalDataSource (couche data)
```

La presentation ne connaît que des abstractions du domaine — entité, cas d'usage et contrat — jamais de datasources, de SQLite ni d'implémentations concrètes. GetIt ne fait que fournir les instances ; le graphe de providers décide des dépendances. C'est pourquoi changer de persistance ou d'implémentation de repository ne change pas une seule ligne de la présentation.

Pourquoi une vente stocke le prix au moment de la vente (snapshot) ?

Chaque SaleItem stocke productName et unitPrice à la date de la vente, en plus de productId. Ne référencer que le produit ne suffirait pas :

1. Variation historique des prix. Les prix des produits changent (promotions, ajustements, inflation). Si la vente ne référençait que le produit, un futur rapport montrerait le prix actuel, pas celui facturé à l'époque — faussant le chiffre d'affaires, les marges et les impôts.
2. Intégrité fiscale et comptable. Une vente est un enregistrement légal : le total facturé ne peut pas changer rétroactivement à cause d'un changement dans la table des prix. Stocker totalAmount + les articles figés garantit cela.
3. Rapports et audit. L'analyse historique (« combien de stylos ai-je vendus en mars ? », « quelle était la marge sur cette période ? ») exige le prix en vigueur à la date de la vente.
4. Évolution du catalogue. Les produits sont renommés, discontinués ou leur id réutilisé. Le snapshot (nom + prix) préserve le vrai contexte de la transaction.
5. Standard du secteur. C'est le comportement classique des ERP/PDV : les *line items* sont la « photographie » de l'article à l'instant de la vente, avec une référence (productId) uniquement pour la traçabilité.

Conclusion : on garde productId pour la traçabilité, mais les valeurs affichées et additionnées (unitPrice, totalAmount) sont celles du moment de la vente — l'historique n'est jamais corrompu par de futurs changements du catalogue.

Comment garantir que le stock ne passe jamais négatif (ProcessSaleUseCase)

ProcessSaleUseCase intègre Ventes, Stock et Finances avec un ordre d'opérations délibéré pour garantir l'intégrité :

1. Validation de la vente — rejette une vente sans articles et un total divergeant de Σ(quantité × prix unitaire), sans rien persister ;
2. Pré-validation du stock — avant toute mutation, vérifie que tous les articles de la vente ont un stock suffisant. Si l'un manque, il lève sans rien persister — il n'y a jamais de vente « orpheline » ;

3. Enregistrement de la vente — ISaleRepository.registerSale ;
4. Décrément du stock — pour chaque article, une diminution qui revalide le stock en interne (défense contre les appels directs) avant de persister la valeur absolue ;
5. Entrée en caisse — enregistre la FinancialTransaction (catégorie « Ventes », paid).

Tout s'exécute dans un `IUnitOfWork` (unit of work) : vente + articles, décrément de stock et entrée en caisse sont écrits dans une seule transaction SQLite — soit tout est persisté, soit rien. BEGIN IMMEDIATE acquiert le verrou d'écriture avant la pré-validation du stock, fermant la fenêtre de concurrence (TOCTOU). En mémoire (web), l'unit of work est un no-op.

UpdateProductStockUseCase est le seul point d'entrée pour muter le stock, et le repository reçoit une valeur absolue (pas un delta) — deux couches de protection : le cas d'usage ne calcule jamais un stock négatif et le repository ne fait que persister. Le CHECK (stock_quantity >= 0) du schéma reste la garde finale.

Comment les couches communiquent sans couplage

ProcessSaleUseCase ne dépend que d'abstractions : le contrat ISaleRepository et UpdateProductStockUseCase (qui dépend du contrat IProductRepository). Aucune d'elles ne connaît les datasources ni SQLite. La composition se produit dans le câblage (ServiceLocator/GetIt), jamais dans les règles métier.

Pourquoi des quantités entières en MILLI-UNITÉS (W11) ?

W4 a retiré double de l'argent (centimes entiers) ; W11 fait de même pour les quantités : 1000 = 1 unité/kg, tout en int.

1. Même philosophie que les centimes. Les balances et PDV au poids produisent des fractions (1.75 kg) ; double accumule des erreurs d'arrondi dans les longues sommes (panier, rapports, diminutions FIFO). Les milli-unités entières rendent l'arithmétique exacte dans chaque couche.
2. Parité Dart ↔ SQL. Il y a un contrat unique : quantité × prix ~/ 1000 (division entière) en Dart et dans les RPC du serveur — totaux du panier, caisse et rapports correspondent octet pour octet.
3. Bordure de conversion concentrée. Quantity (lib/core/utils/quantity.dart) contient les conversions et le formatage (« 1.75 kg ») ; le reste de l'app ne voit que des int.

Pourquoi des lots FIFO avec péremption ?

Les périssables et la péremption sont la réalité quotidienne d'une épicerie : le stock doit sortir dans le bon ordre, sinon le commerçant vend de la marchandise périmée (perte + risque sanitaire).

1. Le lot est l'unité de traçabilité. Chaque entrée en stock crée un StockLot (quantité initiale/actuelle, péremption, coût d'acquisition) — on sait combien reste de chaque entrée et laquelle expire en premier.
2. Diminution FIFO déterministe. FifoStockUseCase (et process_sale sur le serveur) consomme les lots triés par expiry ASC → entry ASC → id ; le surplus passe au lot suivant ; la somme des lots est pré-validée avant toute mutation (pas de diminution partielle).

3. Invariant `stock = Σ lots`. products.stock_quantity est toujours la somme des lots actifs — vérifié par un test (sql_check.sh) et par la requête en lecture seule de sql_check_live.sh. La migration W11 crée un lot initial (lot-mig-…) pour chaque produit avec du stock, donc l'invariant tient dès le premier jour.
4. Alertes de péremption. Le tableau de bord montre rouge (lot périmé avec du stock) et jaune (expire dans < 30 jours) — action préventive avant la perte.

Pourquoi le format balance 20PPPPPWWWWW ?

C'est le format d'étiquette standard du marché pour les balances de prix : préfixe 20, 5 chiffres produit (PLU) et 5 chiffres pour la valeur pesée.

1. Zéro configuration au PDV. La lecture (scanner USB ou caméra) atterrit dans le champ code-barres ; ScaleBarcode reconnaît le motif, extrait le PLU et déduit la fraction poids = valeur ÷ prix — ne surfacture jamais, même quand la balance stocke une valeur monétaire.
2. Repli sûr. Il n'est traité comme un code balance que lorsque le produit existe et est vendaPorFracao ; les codes EAN-12 commençant par 20 (code-barres ordinaires) retombent sur la recherche commune. Les PLU sont normalisés (zéros de tête) dans la recherche.
3. Milli-unités de bout en bout. La fraction entre dans le panier en milli-unités, donc le total, la diminution FIFO et les rapports utilisent tous la même arithmétique entière.

Pourquoi l'échelle typographique suit Apple HIG et Material (34/28/22/20/17/15/13/11) ?

Le TextTheme de l'app n'est pas un ensemble de tailles arbitraires : il suit une échelle modulaire inspirée des Apple Human Interface Guidelines et du Material Design de Google, adaptée à un ERP/PDV de comptoir — où les tablettes sont à quelques centimètres des yeux et la hiérarchie doit être lisible d'un coup d'œil.

L'échelle complète (8 niveaux, mappés aux rôles Material)

Niveau	Rôle Flutter	Taille (sp)	Poids	Où il est utilisé dans l'ERP	tabularFigures
Large Title	displaySmall	34	w700	Écran de connexion, écrans de bienvenue/onboarding, rapports de clôture de caisse consolidés	
Titre 1 (Headline)	headlineMedium	28	w700	Titres de sections principales, en-têtes de page (ex. « Gestion du stock »)	
— intermédiaire	headlineSmall	24	w700	Valeurs KPI et totaux mis en avant sur le tableau de bord	
Titre 2	titleLarge	22	w700	Titres des cartes de résumé, noms de modules, en-têtes de modales, titre AppBar	

Titre 3	titleMedium	20	w600	Sous-titres, en-têtes de sections, noms de produits en vedette dans la liste des ventes	
— intermédiaire	titleSmall	14	w600	Surbrillances de lignes, noms de produits dans les cartes vente/achat	
Corps Large (Headline)	bodyLarge	17	w400	Texte principal des formulaires, champs de saisie, éléments de liste importants	
Corps (Callout)	bodyMedium	15	w400	Descriptions secondaires, notes clients, détails de lots, prix et quantités de lignes	
Footnote	bodySmall	13	w400	Dates de péremption (FIFO), code-barres, unités de mesure, pieds de tableaux	
— boutons/chips	labelLarge / labelMedium	14 / 12	w600	Boutons, chips et actions contextuelles	
Légende	labelSmall	11	w500	Libellés de statut (« En stock », « Périmé »), prix unitaires, petits indicateurs	

Les niveaux en gras sont les huit niveaux de l'échelle Apple/Google ; les lignes restantes sont des rôles Material qui comblent les écarts entre les niveaux (valeurs KPI à 24, libellés de lignes et boutons à 14/12). L'ERP rend délibérément les titres d'un poids plus lourd que le tableau suggéré (w700 sur Titre 1/2 au lieu de w600) — un compromis choisi pour la lisibilité au comptoir, où la hiérarchie doit être saisie d'un coup d'œil pendant que les yeux sont sur le client. tabularFigures est appliqué à chaque style qui porte des nombres () et délibérément omis des styles de libellés ().

Règles d'or appliquées

1. Hiérarchie par proportion, pas par prolifération de tailles. L'échelle définit des niveaux clairs — Large Title 34 → Légende 11 — pour que l'œil trouve instantanément le niveau dont il a besoin (titre d'écran, titre de carte, valeur de ligne, badge) sans scanner une douzaine de tailles.
2. Contraste de poids au lieu de plus de tailles. Le corps du texte reste en trois tailles (17/15/13) ; ce qui distingue une description d'une valeur est le poids (w400 pour le texte, w600 pour les valeurs et libellés importants), la règle d'or Apple/Google. L'ancien thème plafonnait à w800 ; le nouveau plafonne à w700 pour que le gras lise toujours comme *emphase*, pas comme un cri.
3. Alignement numérique (`tabularFigures`). Les prix (MZN) et les quantités (kg/unités) apparaissent en colonnes — rapports, panier, chips de statut. FontFeature.tabularFigures() donne à chaque chiffre la même largeur (largeur tabulaire), donc les colonnes de valeurs s'alignent chiffre par chiffre, façon tableur. Il est appliqué aux styles numériques (displaySmall, headlineMedium/Small, titleLarge/Medium, bodyLarge/Medium/Small) et délibérément pas aux styles de libellés (titleSmall, labelLarge/Medium/Small), qui ne portent pas de chiffres — ni aux styles inline qui portent des nombres (StatusChip labels, avatars de rang de rapports, pilule de comparaison jour-jour), pour que chips et colonnes s'alignent aussi.

4. Lisibilité sur n'importe quel écran. sp signifie *scale-independent pixels* : il respecte l'échelle de texte d'accessibilité de l'OS. La suite valide jusqu'à 2.0× l'échelle de police sans débordement — sur les tablettes de comptoir comme sur les téléphones.
5. Échelle verrouillée par les tests. typography_scale_test.dart audite la taille, le poids et la présence/absence de tabularFigures sur chaque rôle typographique (clair et sombre), plus les surfaces qui suivent l'échelle : AppBar (22/w700), chips (13/w600) et bouton principal (15/w600) — toute régression d'échelle échoue délibérément le test.

Pourquoi une entité financière séparée de la vente ?

FinancialTransaction (entrées/sorties de caisse) est un concept financier ; Sale est un événement opérationnel. Les séparer est délibéré :

1. Toutes les entrées ne viennent pas d'une vente, ni toutes les sorties de la marchandise. Les prêts, apports de capital, intérêts et remboursements sont des entrées ; le loyer, les services publics, les salaires, les impôts et l'entretien sont des sorties qui n'ont ni article, ni stock, ni produit attachés. Les mélanger dans Sale forcerait des champs vides et des règles incohérentes.
2. Les dépenses fixes ont leur propre nature. Elles sont périodiques, récurrentes et catégorisables. Une entité dédiée (description, category, date) permet de répondre *« combien ai-je dépensé en électricité cette année ? »* sans contaminer le modèle des ventes.
3. Vrai flux de trésorerie. Le bon solde = entrées (ventes + autres) − sorties (fixes + variables). Avec une entité financière unique, GetCashFlowUseCase agrège tout sur une période — Sale continue de servir les rapports opérationnels.
4. Responsabilité unique. Sale = opération (articles, stock, paiement) ; FinancialTransaction = argent (sens, catégorie, valeur). Chacun évolue avec ses propres exigences sans casser l'autre.
5. Audit et traçabilité. Chaque mouvement a sa propre identité, catégorie et date, rendant le rapprochement et les rapports par catégorie faciles.

Résumé : la vente enregistre *ce qui s'est passé* (articles et stock) ; la transaction financière enregistre *l'argent qui est entré ou sorti*. Deux modèles, deux responsabilités, un flux de trésorerie exact.

Contrôle du crédit (« fiado ») et fonds de roulement

Le contrôle du crédit répond à la question la plus importante du petit commerçant : qui me doit, combien et quand cela arrive à échéance ?

Structure

- `Debt` (crédit/IOU) — un enregistrement de promesse de paiement avec personName, amount, dueDate et isCustomerDebt :
- isCustomerDebt: true → à recevoir (le client doit au commerçant) ;
- isCustomerDebt: false → à payer (le commerçant doit au fournisseur).
- `GetPendingDebtsUseCase` — filtre et liste uniquement ce qui est à recevoir des clients, répondant immédiatement à *« combien ai-je à recevoir et de qui »*.

- `FinancialTransaction.status` (paid/pending) — sépare l'argent déjà en caisse de la simple entrée attendue. En enregistrant un crédit, AddDebtUseCase crée immédiatement la transaction en attente liée (debtId) : une entrée en attente pour les à recevoir et une sortie en attente pour les à payer. Les transactions en attente peuvent aussi être créées sur l'écran Finances (bascule « Pendente (fiado) ») ; une fois reçues, elles sont marquées paid via le bouton « Receber » (Recevoir) (ou en soldant le crédit) — la caisse réalisée compte l'entrée à la date de réception.
- `Debt.isPaid` — état du règlement : quand le client paie, SettleDebtUseCase marque la dette isPaid: true. Si la dette a une transaction en attente liée, le règlement marque la même transaction comme payée ; sinon il enregistre l'entrée en caisse. Il n'y a jamais de doublon.

Impact sur le fonds de roulement

Les à recevoir sont du capital immobilisé : la marchandise est partie (et a été payée au fournisseur), mais l'argent n'est pas revenu. Avec cette structure, le commerçant peut :

1. Savoir exactement combien « dort » entre les mains des clients — la somme des à recevoir — et la comparer à ce qui est en caisse, évitant un flux de trésorerie surestimé ;
2. Planifier les achats sans dépendre du crédit — si le fonds de roulement est immobilisé, le commerçant peut retarder les achats ou négocier les conditions avec les fournisseurs ;
3. Prioriser les encaissements — dueDate permet de lister ce qui arrive à échéance en premier et d'identifier les défaillants ;
4. Calculer le cycle financier — l'intervalle entre l'achat (à payer), la vente (à crédit) et la réception (à recevoir) définit le fonds de roulement nécessaire pour opérer sans interruption.

Pourquoi l'authentification est-elle la première étape du modèle SaaS ?

1. Multi-tenant dès le premier jour. Chaque commerçant est un *tenant* : sans connexion, impossible de séparer les données de chaque client dans la même base. L'authentification fournit l'identité qui isole les données.
2. Sécurité appliquée dans la base (RLS). Supabase utilise le JWT de l'utilisateur dans le *Row Level Security* Postgres : la règle d'isolation (auth.uid() = user_id OU public.is_my_store(store_id)) est appliquée par la base, pas seulement par l'UI — même si un bug de l'UI fuit des données, la base les filtre.
3. Modèle de revenus. Le SaaS facture par abonnement/forfait ; l'authentification identifie qui paie, quel forfait et quelles limites s'appliquent.
4. Onboarding et rétention. Inscription → connexion → données personnelles ; une session persistante améliore l'expérience et réduit la friction quotidienne.

Comment la session reste active (persistance de session)

1. Stockage sécurisé. supabase_flutter persiste la session via un LocalStorage personnalisé que nous câblons au bootstrap — SecureSupabaseLocalStorage délègue à `flutter_secure_storage` (Keychain/Keystore) sur les plateformes natives (le défaut du SDK utiliserait shared_preferences en texte clair — corrigé lors de l'audit du Aug 6). Le vérificateur de code PKCE utilise le même backend sécurisé. Sur web, le défaut du SDK (window.localStorage) est conservé — flutter_secure_storage 10.x n'a pas de support web.
2. Restauration au démarrage. À l'ouverture de l'app, le flux d'auth ré-émet la session enregistrée. Le AuthGate réagit : session restaurée → directement au tableau de bord, pas de nouvelle connexion.
3. Renouvellement automatique du JWT. GoTrue renouvelle le jeton d'accès avec le jeton de rafraîchissement avant son expiration.
4. Réactivité. Connexion/déconnexion émettent des événements sur le même flux ; la navigation et tout état dépendant de la session réagissent automatiquement.

Les identifiants sont fournis via flavors + `--dart-define-from-file` (env/dev.env commité, env/prod.env gitignoré) ; un build release avec les défauts dev embarqués lève une erreur au démarrage (fail-fast S2 + A1 — tout release exige des identifiants explicites plus le marqueur SUPABASE_CREDENTIALS_EXPLICIT=true).

Devise et langue du tenant (présentation)

Le propriétaire choisit la devise (BRL, USD, EUR, MZN — MZN par défaut, metical mozambicain, le marché cible) et la langue (pt, en, fr) lors de l'onboarding et peut les changer dans Magasins → Réglages. Les préférences vivent dans la table tenants (currency_code, language) et propagent à toute l'app : la locale du MaterialApp et le formatage de devise (Formatters) suivent le tenantSettingsProvider — les employés héritent du choix du propriétaire. L'enregistrement passe par la RPC update_tenant_settings (SECURITY DEFINER, garde is_tenant_owner) : la politique RLS tenants_update ne laisse que le super admin modifier la ligne du tenant, donc un UPDATE direct du propriétaire était bloqué (réglages qui ne s'enregistraient jamais) — la RPC restreint l'UPDATE aux colonnes de présentation.

Stockage sécurisé : pourquoi ne pas utiliser shared_preferences pour les données sensibles ?

SecureStorageService encapsule la persistance chiffrée des jetons d'accès, des jetons de rafraîchissement et des clés API. shared_preferences n'est jamais utilisé pour ce type de données.

Différence technique entre stockage normal et sécurisé

Aspect	shared_preferences (normal)	flutter_secure_storage (sécurisé)
Emplacement physique	Fichier de préférences en texte clair (XML/JSON/plist)	Keychain (iOS/macOS) et Android Keystore
Chiffrement	Aucun — la valeur est lisible sur disque	Données chiffrées ; sur Android, la clé (AES-GCM) est générée et conservée dans le Keystore, liée au matériel

Accès	Tout processus ayant la permission de lecture du fichier	Protégé par l'OS : seule l'app authentifiée y accède ; sur iOS, l'accès peut exiger la biométrie
Sauvegarde/extraction	Les valeurs en clair peuvent être lues depuis les sauvegardes, root/JB ou extraction APK	Chiffré de bout en bout ; sur iOS, elles vivent dans l'enclave sécurisée matérielle
Cas d'usage idéal	Préférences UI (thème, langue, filtres) — rien de sensible	Jetons de session, jetons de rafraîchissement, clés API, secrets

Pourquoi c'est vital pour protéger le compte du commerçant

1. Le jeton est le mot de passe. Avec le jeton de session en main, un attaquant accède au compte comme s'il était le commerçant. Stocker ce jeton en texte clair transforme une sauvegarde, un appareil rooté ou un APK extrait en passerelle vers le compte et ses données financières.
2. Le chiffrement est lié à l'appareil. Sur Android, la clé maîtresse vit dans l'Android Keystore (adossée au matériel, TEE) ; sur iOS, dans la Keychain avec l'enclave sécurisée. Même si le fichier de données est copié, sans la clé matérielle le contenu est inutile.
3. Coût d'attaque élevé. Voler un jeton protégé exige de compromettre l'OS/le matériel — infaisable à l'échelle.
4. Accès conditionné à l'utilisateur (iOS). La Keychain peut exiger Face ID/Touch ID — une barrière de plus avant toute fuite.

Comment le service a été conçu

- Backend abstrait (SecureStorageBackend) : l'implémentation réelle utilise Keychain/Keystore via flutter_secure_storage ; les tests utilisent un backend en mémoire.
- Clés centralisées dans SecureStorageKeys : évite les fautes de frappe et permet d'auditer exactement ce qui est traité comme sensible.

Frontière de responsabilité (un seul backend de stockage, pas de duplication)

La session Supabase est persistée par supabase_flutter via le même backend sécurisé que nous fournissons (SecureSupabaseLocalStorage → flutter_secure_storage) ; le vérificateur PKCE l'utilise aussi (SecureGotrueAsyncStorage). Il y a exactement une couche de stockage pour le matériel d'auth — pas de duplication, pas de texte clair. SecureStorageService sert aussi les secrets d'application : clés API tierces, secrets de configuration et identifiants nécessaires aux couches données/présentation.

Particularité de plateforme (précision technique)

- Android/iOS (mobile) : adossé au matériel — Android Keystore (AES-GCM, clé dans le TEE) et Keychain avec enclave sécurisée. C'est là que les données critiques devraient vivre.
- Windows : DPAPI (lié au compte utilisateur de l'OS) et Linux : libsecret. Protection raisonnable, pas d'enclave dédiée.
- Web : plus faible — l'implémentation web garde la clé de chiffrement dans localStorage (pas adossée au matériel). Pour les secrets critiques, privilégiez le mobile.

Dépendances initiales

Package	Rôle	Couche
flutter_riverpod	Gestion d'état réactive	présentation
flutter_localizations + intl	Internationalisation (pt/en/fr) et formatage	présentation/core
freezed + json_serializable	Modèles immuables et sérialisation JSON (génération de code)	domaine/données
get_it	Injection de dépendances (Service Locator)	core
supabase_flutter	Authentification SaaS, base de données et realtime	données
flutter_secure_storage	Persistance chiffrée des jetons et clés	core
sqlite3 + sqlite3_flutter_libs	Persistance locale relationnelle	données
mobile_scanner	Lecture code-barres caméra au PDV (balances/PLU, W11)	présentation
path_provider	Chemin du dossier Documents	données/core

Génération de code (Freezed)

Chaque fois que vous créez ou modifiez une classe annotée avec @freezed, générez les fichiers :

```
dart run build_runner build --delete-conflicting-outputs
```

Couche de présentation (résumé)

L'app a une navigation adaptative (AppShell avec NavigationRail sur écrans larges et NavigationBar sur écrans étroits) et des modules fonctionnels :

1. Accueil — tableau de bord avec KPI (ventes, solde de caisse par période, à recevoir, stock), actions rapides, ventes récentes et alerte de stock faible ;
2. Produits — recherche, liste avec prix/stock et inscription via bottom sheet ;
3. Ventes — historique et flux de nouvelle vente (panier + moyen de paiement → décrément de stock + entrée en caisse, atomiquement) ;
4. Finances — flux de trésorerie par période (aujourd'hui/7 jours/mois/tout) avec entrées, sorties et solde, transactions et réception des montants en attente ;
5. Crédit (« fiado ») — total à recevoir, dettes par échéance, enregistrement de crédit et règlement avec un historique des règlements ;
6. Magasins — gestion du tenant : magasins, employés, forfait et Réglages (langue + devise du propriétaire) ;
7. Achats — entrée de marchandise avec mise à jour du stock et sortie de caisse.

Le visuel est minimaliste (Material 3) : surfaces neutres, cartes avec bordures subtiles, coins arrondis et formatage de devise localisé (Formatters).